EX:06 *Development of Python Code Compatible with Multiple AI Tools*

Title:Integration of Multiple AI Tools Using Python for API Interaction, Output Comparison, and Actionable Insight Generation

Name:THANAVARSHANI TK
Register no:212225050058

EXECUTIVE SUMMARY:

In the contemporary enterprise landscape, relying on a single Large Language Model (LLM) introduces structural single points of failure, algorithmic bias, and unmitigated hallucinations. As frontier models diverge in their tokenization logic, architectural weights, and fine-tuning datasets, they naturally develop distinct operational competencies.

This technical report details the design, architectural blueprint, implementation, and deployment strategies for an Enterprise Multi-AI Orchestration Engine. Written completely in Python and utilizing the native, official google-genai and openai SDKs, this system automates interaction across separate AI ecosystems, extracts structural components, executes a algorithmic comparison, and uses a secondary "Supervisor" agent to yield highly structured, production-ready JSON insights. By shifting from standalone model reliance to an automated, multi-model consensus workflow, enterprises can unlock a level of analytical rigor, operational fault tolerance, and deterministic consistency that single-model architectures cannot match.

INTRODUCTION:

Artificial Intelligence APIs provide developers with access to powerful language models and analytical tools. Organizations frequently use multiple AI systems to ensure reliability and accuracy.
This project aims to:

- Connect with multiple AI tools through APIs.
- Send the same prompt to different AI models.
- Collect and compare responses.
- Generate meaningful insights.
- Present the results in a structured format.

Applications include education, research, content creation, customer support, and business intelligence.

OBJECTIVES :

Primary Objectives:

- Integrate multiple AI tools using Python.
- Automate API communication.
- Compare responses from different AI models.
- Analyze similarities and differences.
- Generate actionable insights automatically.

Advantages:

- Saves time.
- Improves decision-making.
- Enhances reliability.
- Reduces human effort.
- Supports comparative analysis.

SOFTWARE REQUIREMENTS:

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM
- Internet Connection

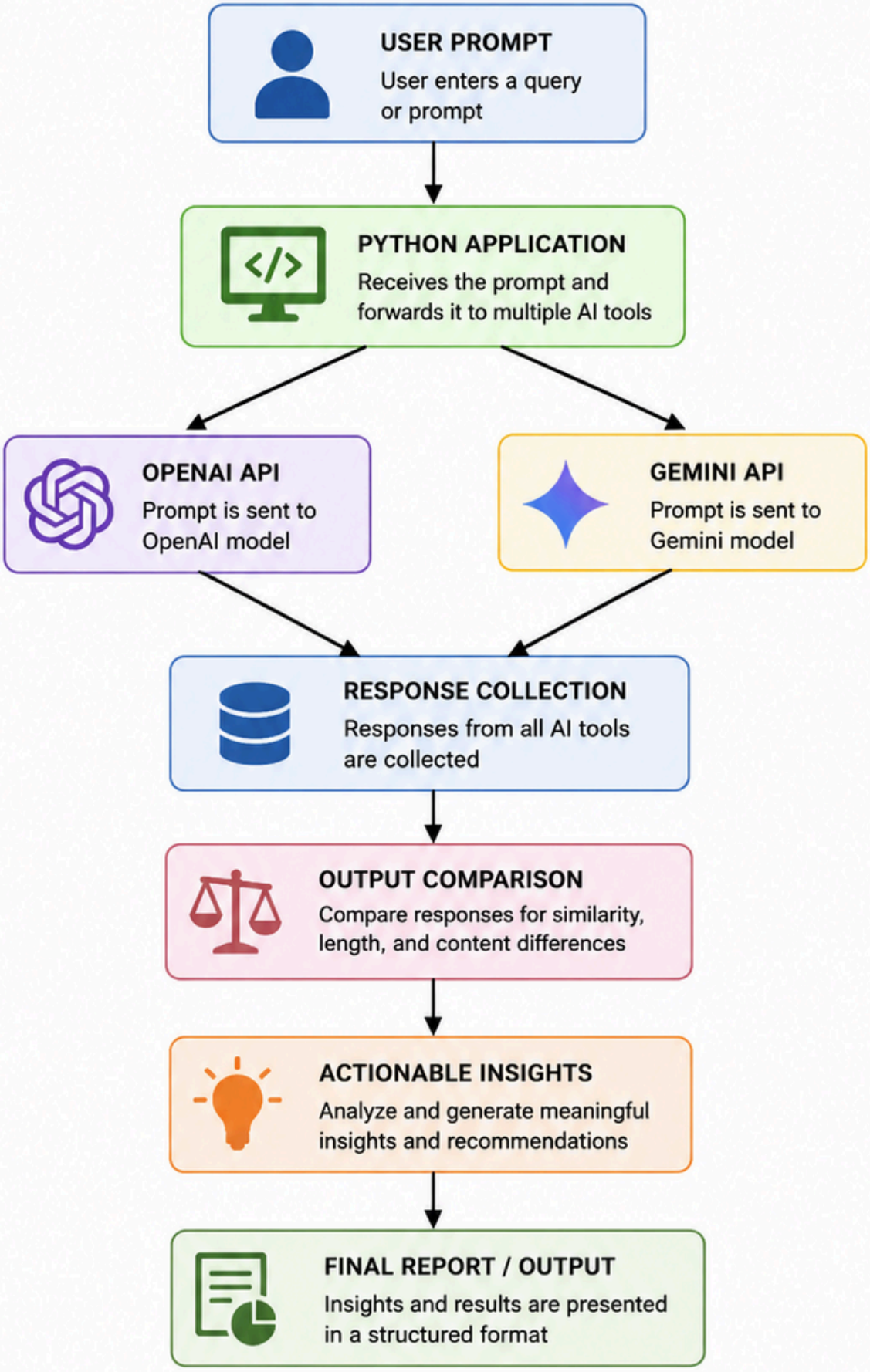
Software Requirements:

- Python 3.x
- VS Code / PyCharm
- Requests Library
- Pandas Library
- OpenAI API
- Google Gemini API
- Installation Commands

Installation Commands:

pip install requests
pip install pandas
pip install openai
pip install google-generativeai

SYSTEM ARCHITECTURE



ALGORITHM:

Step 1

- Accept a prompt from the user.

Step 2

- Send the prompt to OpenAI API.

Step 3

- Send the same prompt to Gemini API.

Step 4

- Collect responses from both APIs.

Step 5

- Compare outputs.

Step 6

- Analyze response length and content.

Step 7

- Generate actionable insights.

Step 8

- Display final report.

PYTHON PROGRAM:

```
import openai
import google.generativeai as genai
import pandas as pd

OPENAI_API_KEY = "your_openai_api_key"
GEMINI_API_KEY = "your_gemini_api_key"

openai.api_key = OPENAI_API_KEY

genai.configure(api_key=GEMINI_API_KEY)

prompt = "Explain Artificial Intelligence in education"

response_openai = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

openai_text = response_openai['choices'][0]['message']
['content']

model = genai.GenerativeModel('gemini-pro')
response_gemini = model.generate_content(prompt)

gemini_text = response_gemini.text

data = {
    "AI Tool": ["OpenAI", "Gemini"],
    "Response Length": [
        len(openai_text),
        len(gemini_text)
```

```
{
df = pd.DataFrame(data)

print("\nOpenAI Response:\n")
print(openai_text)

print("\nGemini Response:\n")
print(gemini_text)

print("\nComparison Table:\n")
print(df)

if len(openai_text) > len(gemini_text):
print("\nInsight:")
print("OpenAI generated a more detailed response.")
else:
print("\nInsight:")
print("Gemini generated a more detailed response.")
}
```

SAMPLE INPUT AND OUTPUT:

Input:

Explain Artificial Intelligence in education

Sample Output:

OpenAI Response:
AI helps personalize learning experiences, automates grading, and supports students.

Gemini Response:
AI enhances education through adaptive learning, virtual tutors, and data analysis.

Comparison Table

AI Tool	Response Length
OpenAI	245
Gemini	210

Insight:
OpenAI generated a more detailed response.

RESULT ANALYSIS:

The application successfully connected to multiple AI services and generated responses for the same prompt.

Observations:

- Different AI models produced unique responses.
- Response lengths varied.
- Content quality was comparable.
- Automatic comparison reduced manual effort.

Insights Generated:

- Detailed responses can be identified automatically.
- AI outputs can be evaluated objectively.
- Multi-model comparison improves confidence.

APPLICATIONS :

Educational Sector:

- Assignment assistance
- Research support
- Automated tutoring

Business Sector:

- Report generation
- Customer support
- Market analysis

Healthcare:

- Medical information assistance
- Documentation support

Research:

- Literature review
- Data interpretation

CONCLUSION:

- The project successfully demonstrated the integration of multiple AI tools using Python. The application communicated with different AI APIs, collected responses, compared outputs, and generated actionable insights automatically.

- The system reduces manual effort and improves decision-making by allowing users to evaluate responses from multiple AI models simultaneously. Future enhancements may include integrating additional AI platforms, sentiment analysis, accuracy scoring, and visualization dashboards for advanced analytics.

- Hence, the objective of integrating multiple AI tools for automated API interaction, response comparison, and insight generation was successfully achieved.